

Homework 7 Report

● Graded

Group

Edison Ricardo Altamirano

Berk Gokmen

[View or edit group](#)

Total Points

100 / 100 pts

Question 1

1

50 / 50 pts

✓ - 0 pts Correct

- 10 pts no discussion of flow creation/modification
- 15 pts no mention of floorplan decisions
- 15 pts no mention of pin assignment decisions

Question 2

2

50 / 50 pts

✓ - 0 pts Met 10ns/0.55 density target with clean timing/DRC/LVS

- 5 pts VDD LVS/DRC errors
- 15 pts Met 20ns/0.3 density target with clean timing/DRC/LVS
- 20 pts Attempted 10ns/0.55 density target with timing/DRC/LVS violations
- 25 pts Attempted 20ns/0.3 density target with timing/DRC/LVS violations
- 2 pts Missing comparison to post-synth results
- 5 pts Missing post-synthesis results
- 40 pts Missing results

EE272 Homework 7
Floorplan, Place & Route, and Signoff
Report

Student Name 1: Berk Gokmen
Student Name 2: Edison Altamirano

- 1) Your strategy/approach. Your report should describe and justify the modifications you made to the existing pipe cleaner place and route flow to make it work for the DNN accelerator. This should include explanation for pin assignment and floor planning decisions. (50 points)

High-Level Strategy

We approached this problem using 0.60 as the target core density in the floorplan for both 20 ns and 10 ns clock period targets. Although achieving the 20 ns target was quite easy, we spent a lot of time fixing errors and violations for the 10 ns clock period. The overall process mainly involved updating two files: *floorplan.tcl* and *pin-assignments.tcl*. We would update these files, read the logs from the signoff and post-route steps in detail, analyze violations, hypothesize how to fix them, update the floorplan and pin assignments again, and iterate. We iterated more than 20 times to achieve a working solution for the 10 ns target.

Initial Floorplan: PlanDesign

For the initial iteration, we used the *planDesign* command to automatically place macros. This produced a simple macro-grouping floorplan:

- 4 KB weight and ifmap macros clumped together on the edge of the chip
- 1 KB ofmap macros arranged in column format, all clumped together
- Uniform halos applied to all macros

While this approach was quick to implement, it produced suboptimal results:

- Long interconnects between data sources and sinks
- Tight channels between macros leading to routing congestion
- DRCs due to limited routing resources

Since the *planDesign* was unsuccessful, we quickly realized that to achieve a good design, we must create a manual floorplan. Figure 1 shows the macro placement produced by *placeDesign*.

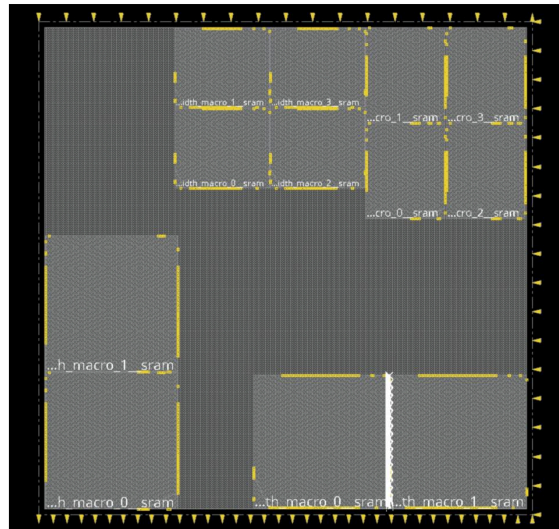


Figure 1. The suboptimal macro placement produced by the *placeDesign* command

Manual Floorplan Placement

After observing the limitations of automatic placement, we transitioned to manual macro placement using *placeInstance* commands in *floorplan.tcl*. The iteration loop followed this workflow:

- Run PnR through the target step
- Inspect congestion, DRC, and max_cap reports
- Shift or rotate macros based on pin access and routing pressure
- Re-run and validate

Macros

We manually placed 12 SRAM instances

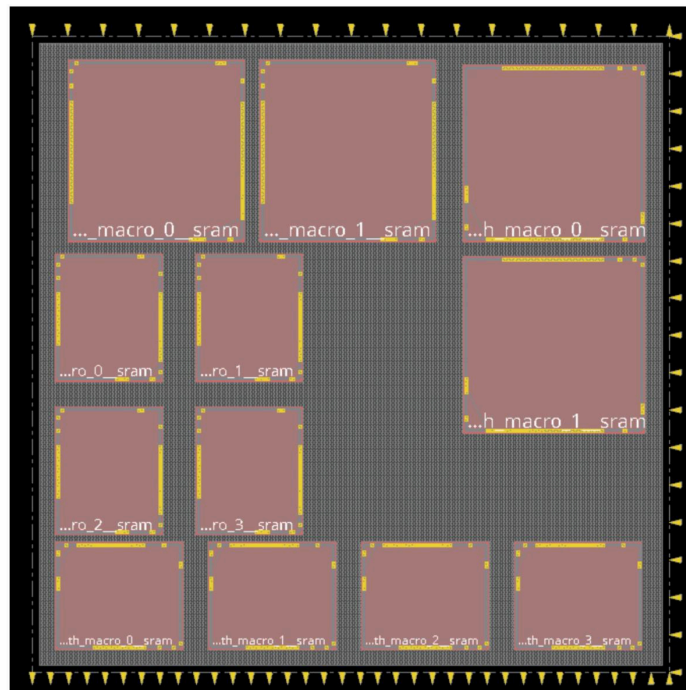
- 2 weight macros (4 KB each)
- 2 ifmap macros (4 KB each)
- 8 ofmap/accumulation macros (1 KB each)

Halo Configuration

We applied a 10.88 um halo to every SRAM macro. This spacing ensured adequate routing channels while not excessively increasing die area. The halo sizes were copied from the pipe-cleaners and never changed as they seemed to work well.

Final SRAM Placement

Instance Name	Coordinates (X, Y)	Orientation
weight_double_buffer_inst/.../depth_macro_0__sram	(150, 1700)	R90
weight_double_buffer_inst/.../depth_macro_1__sram	(900, 1700)	R90
ifmap_double_buffer_inst/.../depth_macro_0__sram	(1700, 1700)	R0
ifmap_double_buffer_inst/.../depth_macro_1__sram	(1700, 950)	R0
ofmap_buffer_inst/ram0/genblk1_width_macro_0__sram	(100, 1150)	R90
ofmap_buffer_inst/ram0/genblk1_width_macro_1__sram	(650, 1150)	R90
ofmap_buffer_inst/ram0/genblk1_width_macro_2__sram	(100, 550)	R90
ofmap_buffer_inst/ram0/genblk1_width_macro_3__sram	(650, 550)	R90
ofmap_buffer_inst/ram1/genblk1_width_macro_0__sram	(100, 100)	MX
ofmap_buffer_inst/ram1/genblk1_width_macro_1__sram	(700, 100)	MX
ofmap_buffer_inst/ram1/genblk1_width_macro_2__sram	(1300, 100)	MX
ofmap_buffer_inst/ram1/genblk1_width_macro_3__sram	(1900, 100)	MX

*Figure 2. Final SRAM Placement*

Sram & Pin Placement Rationale

The final macro floorplan was inspired by the floorplan shown in the review session. Macros were placed around the frame of the chip, as shown in Figure 2, to allow space in the middle for the systolic array. The systolic array accesses all the SRAMs; therefore, it is essential that the routing from the systolic array to the SRAMs is not too long. It is also important that the routing from the chip pins to the SRAMs is not too long. As a result, we had to balance the SRAMs' proximity to the edge of the chip and the center of the chip to achieve the best results.

The ifmap and weight buffers were placed toward the top edge of the chip because they take off-chip inputs (weights and inputs), which are then passed down to the systolic array. On the other hand, the ofmaps are placed toward the bottom because they receive the output of the systolic array from above, and the writeback ofmaps write the final values to external memory from the bottom.

Pin placement was implemented using *editPin* commands in *pin-assignments.tcl*. The strategy followed these principles:

- Place pins close to their associated macro data paths to minimize wire length
- Spread pins along 3 sides to avoid corner congestion
- Pins were arranged evenly within specified regions
- Place the clock pin near the center of one edge to facilitate balanced clock tree distribution

The pin placement can be seen in Figure 2. As shown in the figure, most of the input pins are in the top and right edge, which feed directly to the ifmap and weight buffers. Most of the output pins are at the bottom edge, which feed directly out of the ofmap buffers.

Final Pin Placement

Edge	Metal Layer	Pin Count	Pins
Top	met2	19	ifmap_weight_data[16:0], ifmap_weight_vld, ifmap_weight_rdy
Right	met3	18	config_data[15:0], config_vld, ofmap_rdy
Bottom	met2	36	clk, rst_n, ofmap_data[31:0], ofmap_vld, config_rdy
Left	-	0	(Unused)

Power Strategy

We implemented a custom power strategy specifically designed for a macro-heavy design. The implementation consisted of the following elements:

1. Global PG Connectivity: globalNetConnect to connect VDD/VSS across the design
2. Stdcell Power Rails: srout -nets {VDD VSS} for horizontal stdcell preroutes
3. Core Ring: addRing -type core_rings around the die boundary
4. Block Rings per SRAM: addRing -type block_rings around each macro
5. Power Mesh: addStripe for vertical (met4) and horizontal (met5) stripes
6. Macro PG Hookup: srout -connect blockPin to connect macro pins to the grid

Key Fix: Per-Macro Ring Creation

Initially, we attempted bulk ring creation around all macros simultaneously. This caused inconsistent connections and max_cap violations. The fix was to create rings per macro instance in a loop:

```
None
foreach inst $sram_insts {
    deselectAll
    selectInst $inst
    addRing -nets {VDD VSS} -type block_rings -around selected ...
}
```

This change improved robustness and eliminated DRC/max_cap issues around macros by ensuring each macro received properly connected block rings.

Violations and Fixes

Throughout the implementation, we encountered and resolved several violation types:

Antenna & Dangling Wire Violations

Issue: Dangling antenna wires on VDD/VSS nets where stripes terminated without connection.

Fix: Used editTrim -nets VDD and editTrim -nets VSS to remove residual power grid routing artifacts.

DRC Violations

Issue: Shorts and spacing violations in tight macro channels.

Fix: Increased inter-macro spacing by repositioning and rotating macros.

Max Capacitance Violations

Issue: Long wires from SRAM output pins to nearest logic elements.

Fix: Implemented per-macro ring creation, moved macros to increase space for buffer placement, and adjusted macro orientation.

Connectivity Violations (Known Issue)

Issue: ~1000+ IMPVFC-96 and IMPVFC-200 violations on VSS net.

Status: We were told that these issues are expected and not to be worried about.

Addition of fixdrv Node

To address persistent antenna and DRV issues, we added a custom mflowgen step called cadence-innovus-fixdrv. This node was inserted between postroute and signoff in the flow graph.

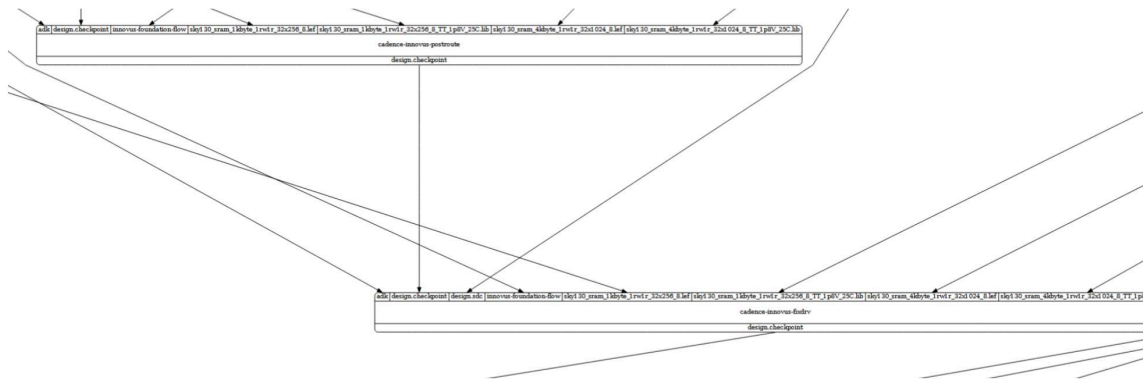


Figure 3. Flow diagram showing fixdrv after postroute (see construct.py for implementation)

The `fixdrv` step executes:

1. editTrim -nets VDD - Remove dangling VDD wires
2. editTrim -nets VSS - Remove dangling VSS wires

We also tried other commands such as fixing glitches, fixing max cap violations, followed by optDesign. However, in the end, we didn't need to call any of these to achieve a working design.

Summary

Our approach followed a systematic iteration pattern:

1. Make a small, targeted change
2. Run the flow to the relevant checkpoint
3. Inspect reports and error logs
4. Identify the next change based on results
5. Repeat until clean

Towards the very end of the project where we were trying to find the optimal floorplan to eliminate the last few errors, we realized that macro placement is extremely sensitive. Moving a macro just by 1-5 micrometers can eliminate many DRC errors or cause many more max cap violations. Towards the end, it was a very trial and error process until we got a design without any issues. Figure 4 below shows the final timing table without any violations besides known issues.

Setup mode	all	default	All2Macro	In2Out	In2Reg	Macro2All	1:Reg..te	Reg2Out	Reg2Reg
WNS (ns):	0.037	0.000	0.063	N/A	0.124	0.037	3.097	6.070	0.069
TNS (ns):	0.000	0.000	0.000	N/A	0.000	0.000	0.000	0.000	0.000
Violating Paths:	0	0	0	N/A	0	0	0	0	0
All Paths:	3438	0	628	N/A	2205	600	267	35	2442
analysis_default	0.037	0.000	0.063	N/A	0.124	0.037	3.097	6.070	0.069
	0.000	0.000	0.000	N/A	0.000	0.000	0.000	0.000	0.000
	0	0	0	N/A	0	0	0	0	0
	3438	0	628	N/A	2205	600	267	35	2442

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	244 (244)	-1.145	244 (244)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 8.609%
(100.000% with Fillers)
Total number of glitch violations: 0

Figure 4. Final signoff summary table, showing no issues are present (excluding known issues)

2) A summary of your design's area, frequency and power and a screenshot of final timing report (setup and hold), verify DRC and verify connectivity. You should compare these to your post-synthesis results. (50 points)

Table of Post-Synthesis vs. Signoff Measurements

	Post-Synthesis	Signoff
Frequency	100 MHz	100 MHz
Area	3557901 μm^2	3582115 μm^2
Total Power	136.513 mW	108.991 mW
Setup WNS	0.028 ns	0.037 ns
Hold WNS	0.301 ns	0.358 ns

We successfully met the 10 ns (100 MHz) timing achieving clean signoff timing with setup WNS = 0.028 ns and hold WNS = 0.3 ns, indicating no timing violations. For density, evaluation from the signoff summary report shows Core Density #2 (excluding physical cells) = 60.231%, which exceeds the 55% requirement, which satisfies the lab's core-density constraint.

The design meets the target frequency of 100 MHz at both post-synthesis and signoff stages, with positive slack on all timing paths. The total cell area increased by 0.68% from synthesis to signoff due to the insertion of timing optimization buffers, clock tree cells, and accounting for actual routing during place and route. Interestingly, total power decreased by 20.1%, from 136.51 mW to 108.99 mW. Although we are not entirely sure why power reduces, this reduction is due to the fact that signoff models more accurate wire capacitance, which is much more accurate than the estimate provided post-synthesis. Both setup and hold WNS improved slightly through the implementation flow. Setup WNS increased from +0.028 ns to +0.037 ns, while hold WNS improved from +0.301 ns to +0.358 ns.

The Figures 5 and 6 below show the timing reports for both hold and setup time, showing that there are no WNS, max_cap, and glitch violations for both cases. The max_tran violations are expected as described in the previous question.

timeDesign Summary									
ID Number		Path Group Name							
1		Reg2ClkGate							
Setup mode	all	default	All2Macro	In2Out	In2Reg	Macro2All	1:Reg..te	Reg2Out	Reg2Reg
WNS (ns):	0.037	0.000	0.063	N/A	0.124	0.037	3.097	6.070	0.069
TNS (ns):	0.000	0.000	0.000	N/A	0.000	0.000	0.000	0.000	0.000
Violating Paths:	0	0	0	N/A	0	0	0	0	0
All Paths:	3438	0	628	N/A	2205	600	267	35	2442
analysis_default	0.037	0.000	0.063	N/A	0.124	0.037	3.097	6.070	0.069
	0.000	0.000	0.000	N/A	0.000	0.000	0.000	0.000	0.000
	0	0	0	N/A	0	0	0	0	0
	3438	0	628	N/A	2205	600	267	35	2442
DRVs		Real		Total					
		Nr nets(terms)	Worst Vio	Nr nets(terms)					
max_cap	0 (0)	0.000	0 (0)						
max_tran	244 (244)	-1.145	244 (244)						
max_fanout	0 (0)	0	0 (0)						
max_length	0 (0)	0	0 (0)						
Density: 8.609%									
(100.000% with Fillers)									
Total number of glitch violations: 0									

Figure 5. Screenshot of setup timing report showing no violations

timeDesign Summary									
ID Number	Path Group Name								
1	Reg2ClkGate								
Hold mode	all	default	All2Macro	In2Out	In2Reg	Macro2All	1:Reg..te	Reg2Out	Reg2Reg
WNS (ns):	0.358	0.000	0.358	N/A	5.136	5.871	0.787	0.776	0.400
TNS (ns):	0.000	0.000	0.000	N/A	0.000	0.000	0.000	0.000	0.000
Violating Paths:	0	0	0	N/A	0	0	0	0	0
All Paths:	3438	0	628	N/A	2205	600	267	35	2442
analysis_default	0.358	0.000	0.358	N/A	5.136	5.871	0.787	0.776	0.400
	0.000	0.000	0.000	N/A	0.000	0.000	0.000	0.000	0.000
	0	0	0	N/A	0	0	0	0	0
	3438	0	628	N/A	2205	600	267	35	2442
Density: 8.609%									
(100.000% with Fillers)									

Figure 6. Screenshot of hold timing report showing no violations

Figure 7 below shows that the target density of 55% is successfully met by our design.

```
ConvVerilog > build-10ns > T9-cadence-innovus-signoff > reports > signoff.summaryReport.rpt
3196 =====
3197 Floorplan/Placement Information
3198 =====
3199 Total area of Standard cells: 2252841.904 um^2
3200 Total area of Standard cells(Subtracting Physical Cells): 186612.726 um^2
3201 Total area of Macros: 3395503.179 um^2
3202 Total area of Blockages: 0.000 um^2
3203 Total area of Pad cells: 0.000 um^2
3204 Total area of Core: 5947303.936 um^2
3205 Total area of Chip: 6231260.022 um^2
3206 Effective Utilization: 1.0000e+00
3207 Number of Cell Rows: 896
3208 % Pure Gate Density #1 (Subtracting BLOCKAGES): 37.880%
3209 % Pure Gate Density #2 (Subtracting BLOCKAGES and Physical Cells): 3.138%
3210 % Pure Gate Density #3 (Subtracting MACROS): 88.284%
3211 % Pure Gate Density #4 (Subtracting MACROS and Physical Cells): 7.313%
3212 % Pure Gate Density #5 (Subtracting MACROS and BLOCKAGES): 88.284%
3213 % Pure Gate Density #6 (Subtracting MACROS and BLOCKAGES for insts are not placed): 7.313%
3214 % Core Density (Counting Std Cells and MACROS): 94.973%
3215 % Core Density #2(Subtracting Physical Cells): 60.231%
3216 % Chip Density (Counting Std Cells and MACROS and IOs): 90.645%
3217 % Chip Density #2(Subtracting Physical Cells): 57.486%
3218 # Macros within 5 sites of IO pad: No
3219 Macro halo defined?: Yes
3220
```

Figure 7. Demonstrating that our design meets the 55%+ requirement

Finally, Figure 8 below shows that our design exhibits no DRC violations, and Figure 9 shows that our design exhibits no connectivity violations (except for those mentioned in the handout as known issues).

```
VERIFY DRC ..... Sub-Area: {1478.400 1330.560 1626.240 1478.400} 164 of 289 Thread : 13
VERIFY DRC ..... Sub-Area: {1182.720 1034.880 1330.560 1182.720} 128 of 289 Thread : 0
VERIFY DRC ..... Sub-Area: {1478.400 1182.720 1626.240 1330.560} 147 of 289 Thread : 13
VERIFY DRC ..... Sub-Area: {1182.720 739.200 1330.560 887.040} 94 of 289 Thread : 12
VERIFY DRC ..... Sub-Area: {1182.720 1182.720 1330.560 1330.560} 145 of 289 Thread : 0
VERIFY DRC ..... Thread : 13 finished.
VERIFY DRC ..... Thread : 14 finished.
VERIFY DRC ..... Thread : 3 finished.
VERIFY DRC ..... Thread : 1 finished.
VERIFY DRC ..... Thread : 7 finished.
VERIFY DRC ..... Sub-Area: {1182.720 887.040 1330.560 1034.880} 111 of 289 Thread : 12
VERIFY DRC ..... Thread : 12 finished.
VERIFY DRC ..... Sub-Area: {1330.560 1330.560 1478.400 1478.400} 163 of 289 Thread : 0
VERIFY DRC ..... Thread : 0 finished.

Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:15.0 ELAPSED TIME: 2.00 MEM: 744.1M) ***
```

Figure 8. Screenshot showing no DRC violations

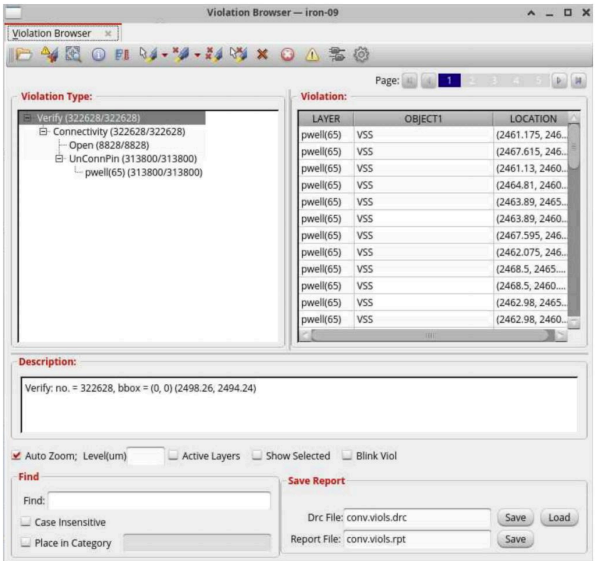


Figure 9. Screenshot showing no connectivity violations (except known issue)

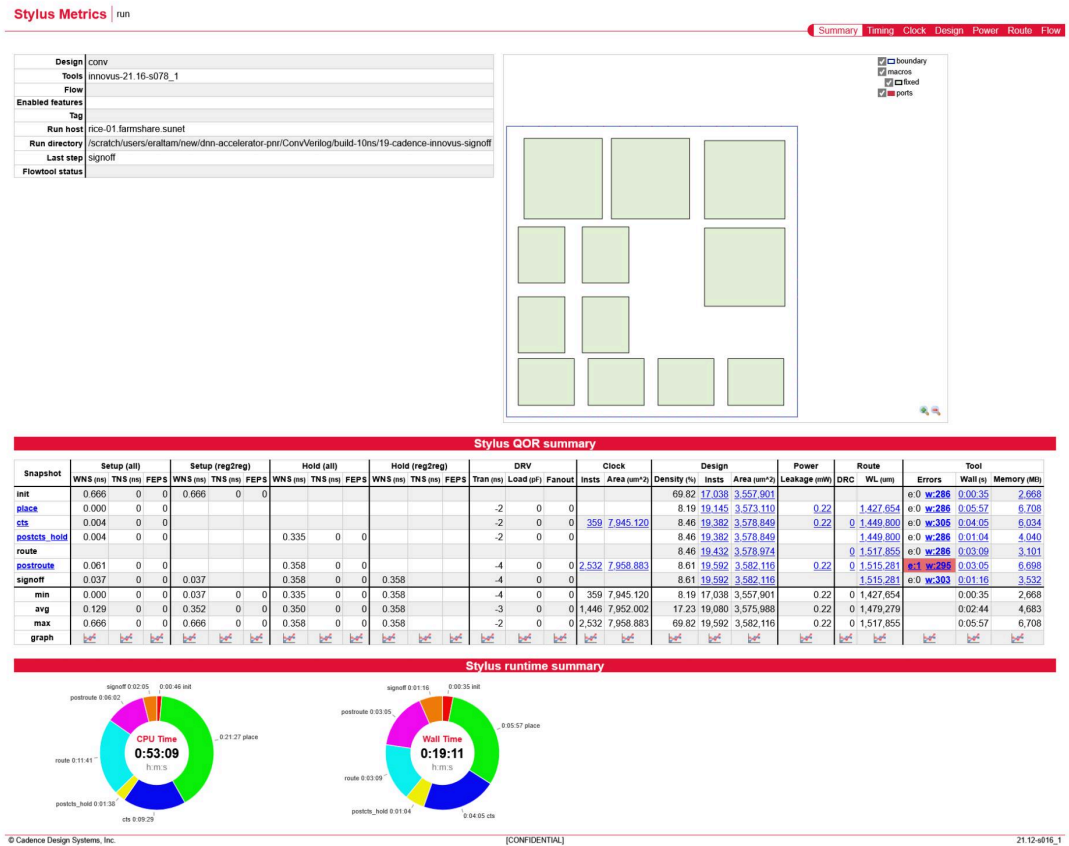


Figure 10. Summary of entire design